<u>Lab Assignment 1A</u>

Due date: 27th January 2021 (3:00PM)

Total Marks: 3

Prakhar Rathi

1810110169

Write a program to implement Quicksort on an array of n integers as follows:

Input the value of n from the user
Randomly generate an array of size n in the range (0, R).
Perform Quicksort on the array to sort where the user selects one of the following options:

i. Choose a random pivot element
ii. Choose first element as pivot point
iii. Choose last element as pivot point
iv. Choose median as pivot point

Output the sorted array

Perform the following experiments:

n	R	Time
1000	**100**	
	1000	
	100000	
10000	**100**	
	1000	
	100000	
100000	**100**	
	1000	
	100000	
1000000	**100**	
	1000	
	100000	

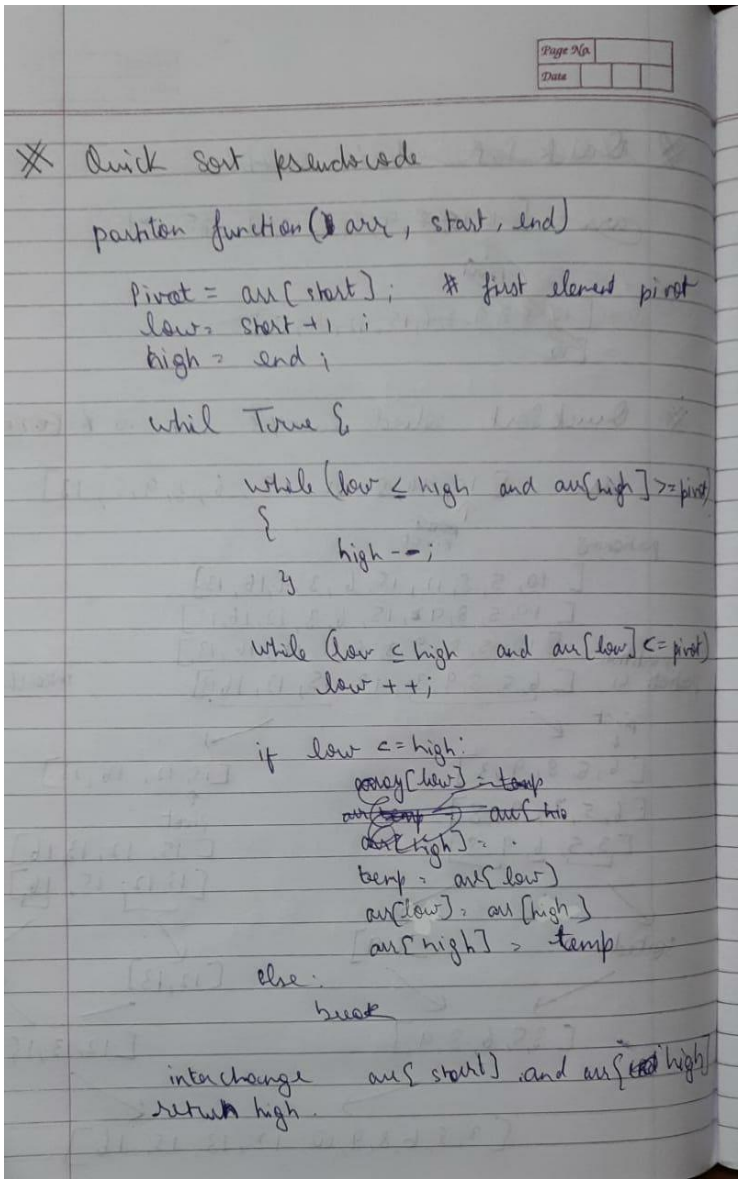
You need to submit the following:

1. A word file containing the following:
    a. Problem statement
    b. Algorithm (in pseudo code)
    c. One solved example (n = 10, Range (0, 20))
    d. Table with results
    e. Result Screenshots

2. Source code – can be found in the lab1.py file

SOLUTION

Algorithm



※ Quick sort pseudocode

```
partition function( arr , start , end)

    Pivot = arr[ start ];   # first element pivot
    low = start + 1 ;
    high = end ;

    whil True {

        while (low ≤ high and arr[high] >= pivot)
        {
            high - - ;
        }

        while (low ≤ high and arr[low] <= pivot)
            low + + ;

        if low <= high:
            array[low] = temp
            arr[high] = arr[low]
            temp = arr[low]
            arr[low] = arr[high]
            arr[high] = temp
        else:
            break

    interchange arr[ start ] and arr[ high ]
    return high.
```

Main function

```
quick_sort ( arr , start, end ) {
    if start >= end
        return
    p = partition ( arr , start , end )
    quick_sort ( arr , start , p-1 )   # quick sort
    quick_sort ( arr , p+1 , end )     # on halves
}
```

One Solved Example

※ Quick Sort solved example n = 10 k = 20

arr = [10, 16, 8, 12, 15, 6, 3, 9, 5, 13]

pivot

partioning

[10, 5, 8, 12, 15, 6, 3, 9, 16, 13]

[10, 5, 8, 9, 15, 6, 3, 12, 16, 13]

[10, 5, 8, 9, 3, 6, 15, 12, 16, 13]

partition ↳ [6, 5, 8, 9, 3, 10, 15, 12, 16, 13] interchange

pivot

[6, 5, 8, 9, 3]

[6, 5, 3, 9, 8]

[3, 5, 6, 9, 8]

sorted

[8, 9]

[3, 5, 6, 8, 9]

[15, 12, 16, 13]

pivot

[15, 12, 13, 16]

[13, 12, 15, 16]

[12, 13]

sorted

[12, 13, 15, 16]

[3, 5, 6, 8, 9, 10, 12, 13, 15, 16]

Results Table

n	R	Time Consumed by the algorithm (seconds)			
		Random Pivot	First Element Pivot	Last Element Pivot	Median as Pivot
1,000	100	0.01301	0.00696	0.010994	0.03999
	1,000	0.01101	0.00701	0.010976	0.00801
	100,000	0.01201	0.008028	0.155498	0.01599
10,000	100	0.20488	0.177899	0.446996	1.66889
	1,000	0.13192	0.089936	0.131448	0.40796
	100,000	0.11894	0.090944	0.102595	0.18398
100,000	100	13.3244	12.54026	49.05915	93.6349
	1,000	2.24671	1.923895	5.184330	12.11451
	100,000	1.29909	1.05739	1.167545	3.059949
1,000,000	100	Max Depth reached	Max Depth reached	Max Depth reached	Max Depth reached
	1,000	135.255	128.78739	634.5688	Max Depth reached
	100,000	19.4501	15.8362	18.19928	32.43072

Results Screenshots

For the result outputs, I am not printing the array because it takes up a lot of space in case of 1000 numbers. From 10,000 onwards, I will print it. However, in the final submission, result outputs will be activated for all the cases and can be tested. Some codes were only working in a jupyter notebook with GPU support. They are pasted in a white cell and can be found inside the jupyter notebook. These codes led to a “Maximum Recursion Depth reached error” in the terminal.

Random Element Pivot

```
Enter the array size (n):1000
Enter the maximum possible value in the array (R): 100
Select the pivot choice from the 4 options above:1

Pivot chosen as a random element
Problem parameters:
n= 1000
R= 100

Original Array: Final Array:
Runtime of the algorithm is 0.013009309768676758 seconds
```

```
C:\Users\hp\Desktop\github_repos\CSD332-Algorithms-Coursework>python lab1.py


                        This is a customized quicksort program.


You have the following options:


                                1.Choose a random pivot element
                                2.Choose first element as pivot point
                                3.Choose last element as pivot point
                                4.Choose median as pivot point

Enter the array size (n):1000
Enter the maximum possible value in the array (R): 1000
Select the pivot choice from the 4 options above:1

Pivot chosen as a random element
Problem parameters:
n= 1000
R= 1000

Original Array: Final Array:
Runtime of the algorithm is 0.01101064682006836 seconds
```

```
Enter the array size (n):1000
Enter the maximum possible value in the array (R): 100_000
Select the pivot choice from the 4 options above:1

Pivot chosen as a random element
Problem parameters:
n= 1000
R= 100000

Original Array: Final Array:
Runtime of the algorithm is 0.012011528015136719 seconds
```

```
Enter the array size (n):10000
Enter the maximum possible value in the array (R): 100
Select the pivot choice from the 4 options above:1

Pivot chosen as a random element
Problem parameters:
n= 10000
R= 100

Original Array: array([63,  1, 10, ..., 41,  3, 71])
Final Array: array([ 0,  0,  0, ..., 99, 99, 99])

Runtime of the algorithm is 0.20488381385803223 seconds
```

```
Enter the array size (n):10000
Enter the maximum possible value in the array (R): 1000
Select the pivot choice from the 4 options above:1

Pivot chosen as a random element
Problem parameters:
n= 10000
R= 1000

Original Array: array([366, 407, 704, ..., 281, 932, 411])
Final Array: array([  0,   0,   0, ..., 999, 999, 999])

Runtime of the algorithm is 0.13192367553710938 seconds
```

```
Enter the array size (n):10000
Enter the maximum possible value in the array (R): 100_000
Select the pivot choice from the 4 options above:1

Pivot chosen as a random element
Problem parameters:
n= 10000
R= 100000

Original Array: array([80276, 67168, 65226, ..., 44373,  4145,  9298])
Final Array: array([    2,    51,    68, ..., 99977, 99981, 99991])

Runtime of the algorithm is 0.11894989013671875 seconds
```

```python
# Call the sorting function
quick_sort(n=100_000, R=100, pivot_option=1)
```

```
Pivot chosen as a random element
Problem parameters:
n= 100000
R= 100

Original Array: array([86, 38, 66, ..., 60, 90,  3])
Final Array: array([ 0,  0,  0, ..., 99, 99, 99])

Runtime of the algorithm is 13.324411869049072 seconds
```

```
Enter the array size (n):100_000
Enter the maximum possible value in the array (R): 1000
Select the pivot choice from the 4 options above:1

Pivot chosen as a random element
Problem parameters:
n= 100000
R= 1000

Original Array: array([464, 266, 248, ...,  10, 439, 440])
Final Array: array([  0,   0,   0, ..., 999, 999, 999])

Runtime of the algorithm is 2.2467153072357178 seconds
```

```
Enter the array size (n):100_000
Enter the maximum possible value in the array (R): 100_000
Select the pivot choice from the 4 options above:1

Pivot chosen as a random element
Problem parameters:
n= 100000
R= 100000

Original Array: array([  419, 50361, 88073, ..., 96837, 37542, 84477])
Final Array: array([    2,     2,     3, ..., 99996, 99998, 99999])

Runtime of the algorithm is 1.2990915775299072 seconds
```

```python
# Call the sorting function
quick_sort(n=1_000_000, R=1000, pivot_option=1)
```

```
Pivot chosen as a random element
Problem parameters:
n= 1000000
R= 1000

Original Array: array([617, 636, 153, ..., 419,  28, 253])
Final Array: array([  0,   0,   0, ..., 999, 999, 999])

Runtime of the algorithm is 135.2557737827301 seconds
```

```
Enter the array size (n):1_000_000
Enter the maximum possible value in the array (R): 100_000
Select the pivot choice from the 4 options above:1

Pivot chosen as a random element
Problem parameters:
n= 1000000
R= 100000

Original Array: array([53508, 76959, 96357, ..., 94228, 27057, 99566])
Final Array: array([    0,     0,     0, ..., 99999, 99999, 99999])

Runtime of the algorithm is 19.450133085250854 seconds
```

First element pivot

```
C:\Users\hp\Desktop\github_repos\CSD332-Algorithms-Coursework>python lab1.py


                        This is a customized quicksort program.


You have the following options:


                                1.Choose a random pivot element
                                2.Choose first element as pivot point
                                3.Choose last element as pivot point
                                4.Choose median as pivot point

Enter the array size (n):1000
Enter the maximum possible value in the array (R): 100
Select the pivot choice from the 4 options above:2
Pivot Chosen as the first element
Problem parameters:
n= 1000
R= 100

Original Array: Final Array:
Runtime of the algorithm is 0.006958723068237305 seconds
```

```
Enter the array size (n):1000
Enter the maximum possible value in the array (R): 1000
Select the pivot choice from the 4 options above:2
Pivot Chosen as the first element
Problem parameters:
n= 1000
R= 1000

Original Array: Final Array:
Runtime of the algorithm is 0.007012367248535156 seconds
```

```
Enter the array size (n):1000
Enter the maximum possible value in the array (R): 100000
Select the pivot choice from the 4 options above:2
Pivot Chosen as the first element
Problem parameters:
n= 1000
R= 100000

Original Array: Final Array:
Runtime of the algorithm is 0.008028507232666016 seconds
```

```
Enter the array size (n):10000
Enter the maximum possible value in the array (R): 100
Select the pivot choice from the 4 options above:2
Pivot Chosen as the first element
Problem parameters:
n= 10000
R= 100

Original Array: array([26, 99, 73, ..., 87, 98, 13])
Final Array: array([ 0,  0,  0, ..., 99, 99, 99])

Runtime of the algorithm is 0.17789936065673828 seconds
```

```
Enter the array size (n):10000
Enter the maximum possible value in the array (R): 1000
Select the pivot choice from the 4 options above:2
Pivot Chosen as the first element
Problem parameters:
n= 10000
R= 1000

Original Array: array([717, 750,  85, ..., 903, 780, 140])
Final Array: array([  0,   0,   0, ..., 999, 999, 999])

Runtime of the algorithm is 0.08993649482727051 seconds
```

```
Enter the array size (n):10000
Enter the maximum possible value in the array (R): 100000
Select the pivot choice from the 4 options above:2
Pivot Chosen as the first element
Problem parameters:
n= 10000
R= 100000

Original Array: array([43707, 19671, 63585, ..., 19105, 80438, 92809])
Final Array: array([    5,    29,    30, ..., 99976, 99991, 99993])

Runtime of the algorithm is 0.09094476699829102 seconds
```

```python
# Call the sorting function
quick_sort(n=100_000, R=100, pivot_option=2)
```

```
Pivot Chosen as the first element
Problem parameters:
n= 100000
R= 100

Original Array: array([98, 20, 58, ..., 90, 71, 67])
Final Array: array([ 0,  0,  0, ..., 99, 99, 99])

Runtime of the algorithm is 12.540265560150146 seconds
```

```
Enter the array size (n):100_000
Enter the maximum possible value in the array (R): 1000
Select the pivot choice from the 4 options above:2

Pivot Chosen as the first element
Problem parameters:
n= 100000
R= 1000

Original Array: array([ 48, 653, 571, ..., 510, 923, 576])
Final Array: array([  0,   0,   0, ..., 999, 999, 999])

Runtime of the algorithm is 1.9238955974578857 seconds
```

```
Enter the array size (n):100_000
Enter the maximum possible value in the array (R): 100000
Select the pivot choice from the 4 options above:2

Pivot Chosen as the first element
Problem parameters:
n= 100000
R= 100000

Original Array: array([78702, 49384, 29207, ..., 60041, 70952, 97077])
Final Array: array([    1,     5,     6, ..., 99998, 99998, 99999])

Runtime of the algorithm is 1.0573906898498535 seconds
```

```python
# Call the sorting function
quick_sort(n=1000000, R=1000, pivot_option=2)
```

```
Pivot Chosen as the first element
Problem parameters:
n= 1000000
R= 1000

Original Array: array([520, 326, 100, ..., 837, 970, 699])
Final Array: array([  0,   0,   0, ..., 999, 999, 999])

Runtime of the algorithm is 128.78739142417908 seconds
```

```
Enter the array size (n):1_000_000
Enter the maximum possible value in the array (R): 100_000
Select the pivot choice from the 4 options above:2

Pivot Chosen as the first element
Problem parameters:
n= 1000000
R= 100000

Original Array: array([92775, 91391, 41067, ..., 98987, 65866,  7572])
Final Array: array([    0,     0,     0, ..., 99999, 99999, 99999])

Runtime of the algorithm is 15.836233139038086 seconds
```

Last element Pivots

```
Enter the array size (n):1000
Enter the maximum possible value in the array (R): 100
Select the pivot choice from the 4 options above:3
Pivot Chosen as the last element
Problem parameters:
n= 1000
R= 100

Original Array: Final Array:
Runtime of the algorithm is 0.010994911193847656 seconds
```

```
Enter the array size (n):1000
Enter the maximum possible value in the array (R): 1000
Select the pivot choice from the 4 options above:3
Pivot Chosen as the last element
Problem parameters:
n= 1000
R= 1000

Original Array: Final Array:
Runtime of the algorithm is 0.01097559928894043 seconds
```

```
Enter the array size (n):1000
Enter the maximum possible value in the array (R): 100000
Select the pivot choice from the 4 options above:3
Pivot Chosen as the last element
Problem parameters:
n= 1000
R= 100000

Original Array: Final Array:
Runtime of the algorithm is 0.015549898147583008 seconds
```

```
Enter the array size (n):10000
Enter the maximum possible value in the array (R): 100
Select the pivot choice from the 4 options above:3

Pivot Chosen as the last element
Problem parameters:
n= 10000
R= 100

Original Array: array([92,  6, 73, ..., 36, 52, 43])
Final Array: array([ 0,  0,  0, ..., 99, 99, 99])

Runtime of the algorithm is 0.44699668884277344 seconds
```

```
Enter the array size (n):10000
Enter the maximum possible value in the array (R): 1000
Select the pivot choice from the 4 options above:3

Pivot Chosen as the last element
Problem parameters:
n= 10000
R= 1000

Original Array: array([693, 471, 675, ..., 223, 230, 697])
Final Array: array([  0,   0,   0, ..., 999, 999, 999])

Runtime of the algorithm is 0.13144803047180176 seconds
```

```
Enter the array size (n):10000
Enter the maximum possible value in the array (R): 100_000
Select the pivot choice from the 4 options above:3

Pivot Chosen as the last element
Problem parameters:
n= 10000
R= 100000

Original Array: array([ 7452, 62657, 15167, ..., 83470, 78774, 95944])
Final Array: array([   23,    28,    40, ..., 99958, 99961, 99991])

Runtime of the algorithm is 0.10259580612182617 seconds
```

```
# Call the sorting function
quick_sort(n=100_000, R=100, pivot_option=3)
```

```
Pivot Chosen as the last element
Problem parameters:
n= 100000
R= 100

Original Array: array([ 2, 47, 97, ...,  4, 76, 48])
Final Array: array([ 0,  0,  0, ..., 99, 99, 99])

Runtime of the algorithm is 42.059154748916626 seconds
```

```
Enter the array size (n):100_000
Enter the maximum possible value in the array (R): 1000
Select the pivot choice from the 4 options above:3

Pivot Chosen as the last element
Problem parameters:
n= 100000
R= 1000

Original Array: array([910, 824, 685, ...,  44, 731, 905])
Final Array: array([  0,   0,   0, ..., 999, 999, 999])

Runtime of the algorithm is 5.18483304977417 seconds
```

```
Enter the array size (n):100_000
Enter the maximum possible value in the array (R): 100_000
Select the pivot choice from the 4 options above:3

Pivot Chosen as the last element
Problem parameters:
n= 100000
R= 100000

Original Array: array([22501, 50477, 90677, ..., 28415, 58803,  5616])
Final Array: array([    0,     0,     0, ..., 99998, 99998, 99999])

Runtime of the algorithm is 1.1675455570220947 seconds
```

```python
# Call the sorting function
quick_sort(n=1_000_000, R=1000, pivot_option=3)
```

```
Pivot Chosen as the last element
Problem parameters:
n= 1000000
R= 1000

Original Array: array([ 56, 714, 335, ...,   0, 405, 248])
Final Array: array([  0,   0,   0, ..., 999, 999, 999])

Runtime of the algorithm is 634.5688211917877 seconds
```

```
Enter the array size (n):1_000_000
Enter the maximum possible value in the array (R): 100_000
Select the pivot choice from the 4 options above:3

Pivot Chosen as the last element
Problem parameters:
n= 1000000
R= 100000

Original Array: array([77695, 79824, 27233, ..., 20677,   618, 45416])
Final Array: array([    0,     0,     0, ..., 99999, 99999, 99999])

Runtime of the algorithm is 18.199280977249146 seconds
```

Median Element Pivot

(Implementation can be found in the jupyter notebook) – Outputs can also be seen there.

```python
N_values = [1000, 10_000, 100_000, 1_000_000]
R_values = [100, 1000, 100_000]

for n in N_values:
    for r in R_values:
        if n == 1_000_000 and (r==100 or r==1000):
            print("Maximum Recursion Depth")
            continue

        # Generate an array of random numbers
        array = np.random.randint(low=0,
                                  high=r,
                                  size=n,
                                  dtype=int)
        random.seed(time.time())

        isMedian = False

        while not isMedian:
            arr_copy = array.copy()
            sub_len = len(arr_copy)
            rand_ind = random.randint(0, sub_len)

            arr_copy[rand_ind], arr_copy[sub_len-1] = arr_copy[sub_len-1], arr_copy[rand_ind]
            pi = partition_med(arr_copy, 0, sub_len-1)

            if 0.45*sub_len < pi < 0.55*sub_len:
                isMedian = True
                arr_copy[rand_ind], arr_copy[sub_len-1] = arr_copy[sub_len-1], arr_copy[rand_ind]
                break

            # Start timing
            start = time.time()
            quick_sort_med(arr_copy, 0, sub_len-1)
            end = time.time()
            total = end - start

            print("N=", n)
            print("R=", r)
            print("Time taken:", end-start)
```